

Week 5 Assignment

Deep Learning & Neural Networks

AI Internship Program | Total Marks: 100

Task	Marks
Task 1: Deep Learning Fundamentals	20
Task 2: Build a Basic Neural Network	20
Task 3: Train & Visualize the Model	20
Task 4: Model Evaluation & Prediction	20
Task 5: Mini Project	20
Total	100

Dataset sources: Iris and Breast Cancer Wisconsin datasets, both standard Scikit-learn built-in datasets. Full code and output for Tasks 2-4 are in Week5_NeuralNetwork.ipynb; the complete Mini Project is in Mini_Project_Breast_Cancer.ipynb.

Task 1: Deep Learning Fundamentals

What is Deep Learning?

Deep Learning is a subset of Machine Learning that uses artificial neural networks with multiple layers (hence 'deep') to automatically learn patterns directly from large amounts of data. Unlike traditional ML techniques that often require manually engineered features, deep learning models learn useful feature representations on their own, which makes them especially powerful for complex, unstructured data like images, audio, and text.

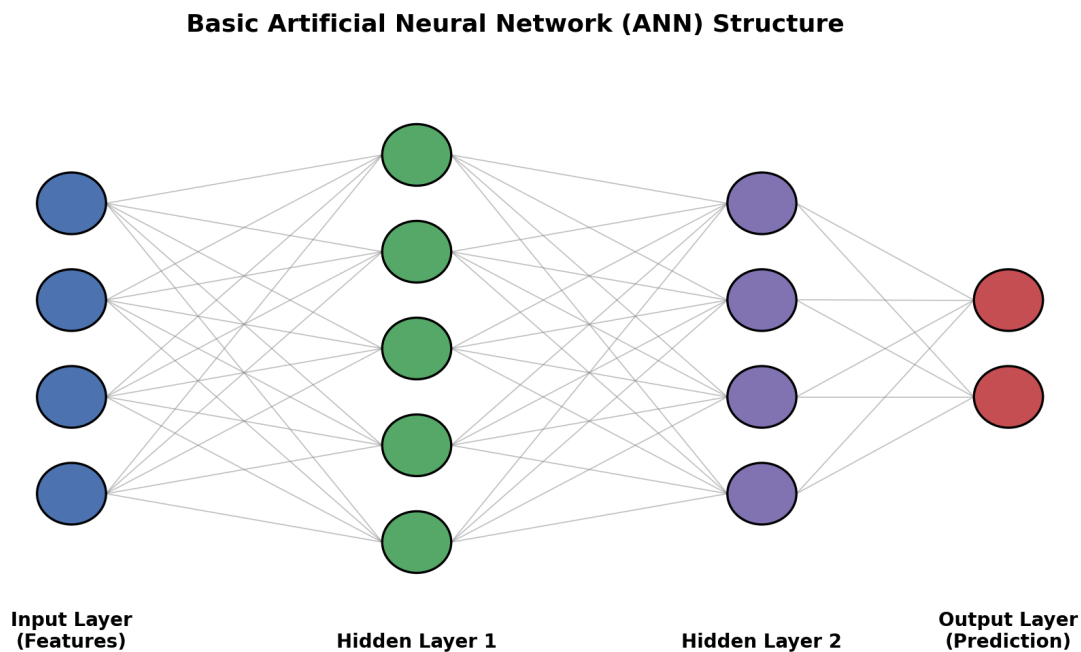
AI vs Machine Learning vs Deep Learning

Artificial Intelligence is the broadest field, covering any technique that makes machines behave intelligently. Machine Learning is a subset of AI where systems learn from data rather than following fixed rules. Deep Learning is a further subset of ML that specifically uses multi-layered neural networks, and is especially effective for tasks with large, complex datasets such as image recognition and natural language processing.

What is an Artificial Neural Network (ANN)?

An Artificial Neural Network is a computing system loosely inspired by the structure of the human brain, made up of layers of interconnected nodes ('neurons') that process information and pass it forward through the network. Each connection between neurons has an associated weight, which the network adjusts during training to improve its predictions.

Diagram of a Basic Neural Network



Key Building Blocks

Term	Explanation
Neuron	The basic computational unit of a neural network; it receives inputs, multiplies them by weights, adds a bias, and produces an output.
Input Layer	The first layer of the network, which receives the raw feature values (e.g. the 4 Iris measurements).
Hidden Layer	Layer(s) between the input and output that transform the data, learning intermediate patterns and representations.
Output Layer	The final layer, which produces the network's prediction (e.g. a class probability).
Weights and Bias	Weights control how strongly each input influences a neuron's output; the bias is an extra adjustable value added to the weighted sum.
Activation Function	A function (e.g. ReLU, sigmoid, softmax) applied to a neuron's output, introducing non-linearity so the network can learn complex patterns.
Epoch	One complete pass of the entire training dataset through the network during training.
Loss	A number measuring how far the network's predictions are from the actual correct answers; training aims to minimize the loss.
Optimizer	The algorithm (e.g. Adam, SGD) that adjusts the network's weights and biases to reduce the loss after each training step.

Basic Working Flow

Input → Neural Network → Prediction → Loss → Optimization → Improved Model.

Data is fed into the network's input layer as numeric features. This data passes forward through the hidden layers, where each neuron applies its weights, bias, and activation function, ultimately producing a prediction at the output layer. This prediction is compared to the actual correct answer using a loss function, which quantifies how wrong the prediction was. The optimizer then uses this loss value to adjust every weight and bias in the network slightly (via a process called backpropagation), nudging the network toward better predictions. This entire cycle repeats over many epochs, with the loss gradually decreasing and the model's predictions gradually improving.

Task 2: Build a Basic Neural Network

Dataset: The Iris dataset (iris.csv) — 150 flower samples across 3 species, with 4 numeric features (sepal length/width, petal length/width).

The dataset was loaded, features and target were separated, and features were scaled using StandardScaler while the target species were one-hot encoded (since this is a 3-class problem). The data was split 80/20 into training and testing sets, stratified by species.

Model Architecture

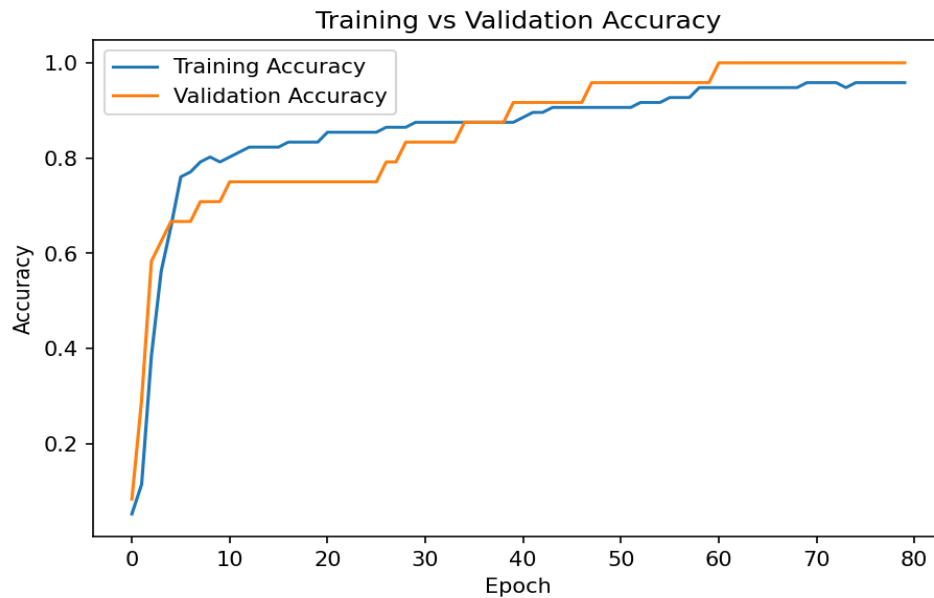
Layer	Neurons	Activation
Input	4 (features)	-
Hidden Layer 1	16	ReLU
Hidden Layer 2	8	ReLU
Output	3 (classes)	Softmax

The model was compiled with the Adam optimizer, categorical crossentropy loss (standard for multi-class classification), and accuracy as the tracked metric. See **Week5_NeuralNetwork.ipynb** for the full code and model summary output.

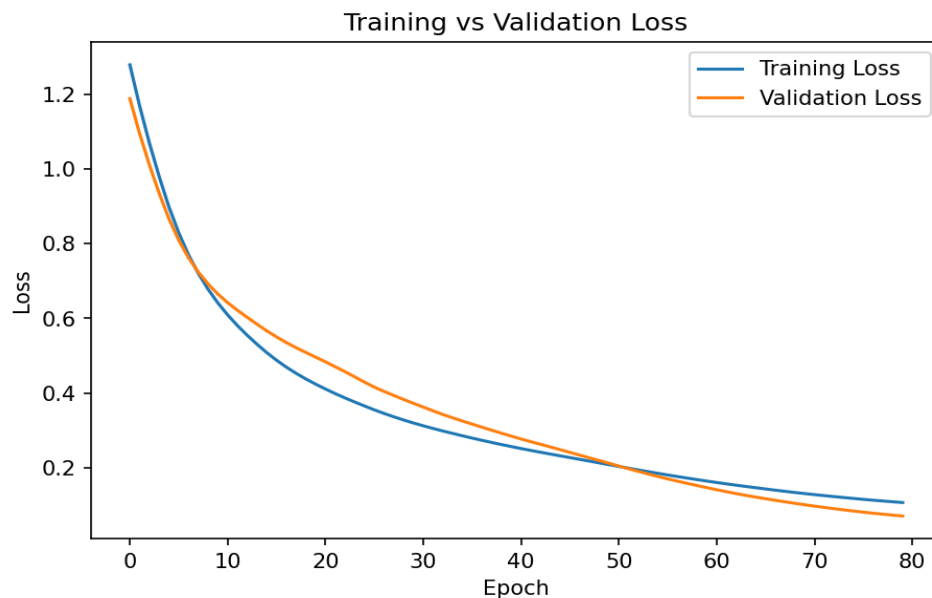
Task 3: Train & Visualize the Model

The model was trained for 80 epochs with a batch size of 8, using 20% of the training data as a validation set to monitor performance during training.

Training vs Validation Accuracy



Training vs Validation Loss



Explanation of Training Behavior

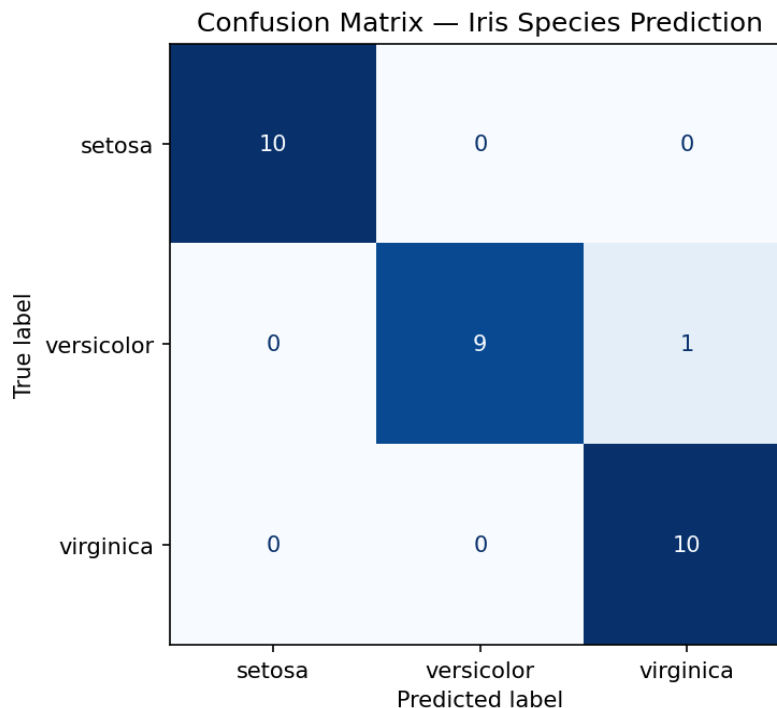
During training, both training and validation accuracy rose sharply over the first 10-20 epochs and then leveled off at a high value, reaching 95.83% training accuracy and 100% validation accuracy by the final epoch. Loss showed the mirror pattern: a steep drop early in training followed by a plateau near zero for

both training and validation loss. The model is clearly improving throughout training, since both metrics move in the right direction (accuracy up, loss down) and largely track each other. There is little to no visible sign of overfitting here, since training and validation curves stay close together rather than training accuracy climbing while validation accuracy stagnates or drops — if a gap like that had opened up, with training loss continuing to fall while validation loss started rising again, that would indicate overfitting.

Task 4: Model Evaluation & Prediction

The trained model was evaluated on the held-out test set (30 samples), achieving a **test accuracy of 96.67%** and test loss of 0.1289.

Confusion Matrix



Actual	Predicted
setosa	setosa (10/10 correct)
versicolor	versicolor (9/10 correct)
virginica	virginica (10/10 correct)

Explanation of Incorrect Predictions

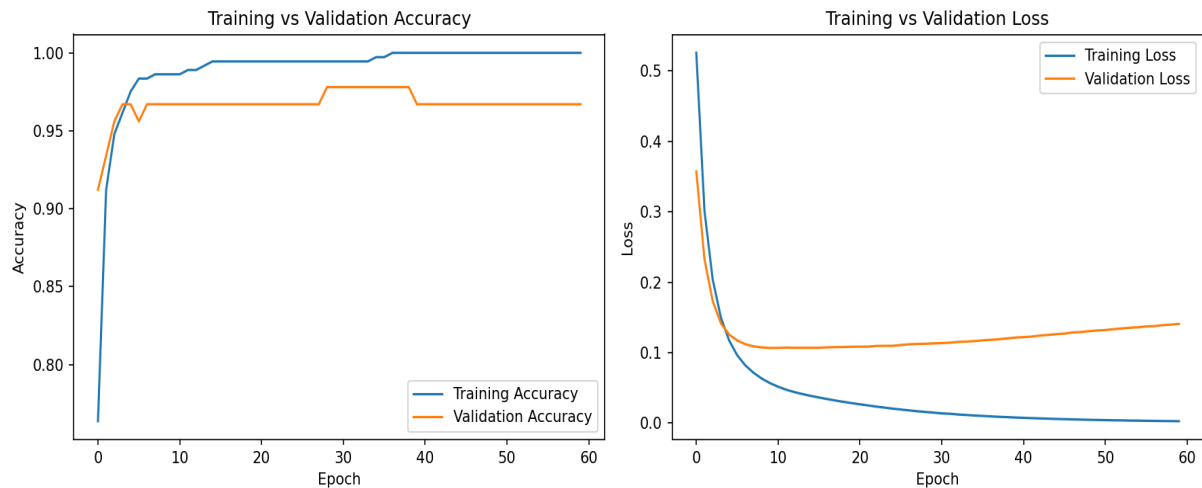
Out of 30 test samples, the model made exactly 1 misclassification: a versicolor sample was predicted as virginica. This is the single incorrect prediction shown off the diagonal in the confusion matrix. This kind of error is common on the Iris dataset, since versicolor and virginica have overlapping petal length and width measurements, unlike setosa which is clearly separated from the other two species by petal size. All 10 setosa and 10 virginica test samples were classified correctly.

See [Week5_NeuralNetwork.ipynb](#) for the full predictions table (10+ examples shown).

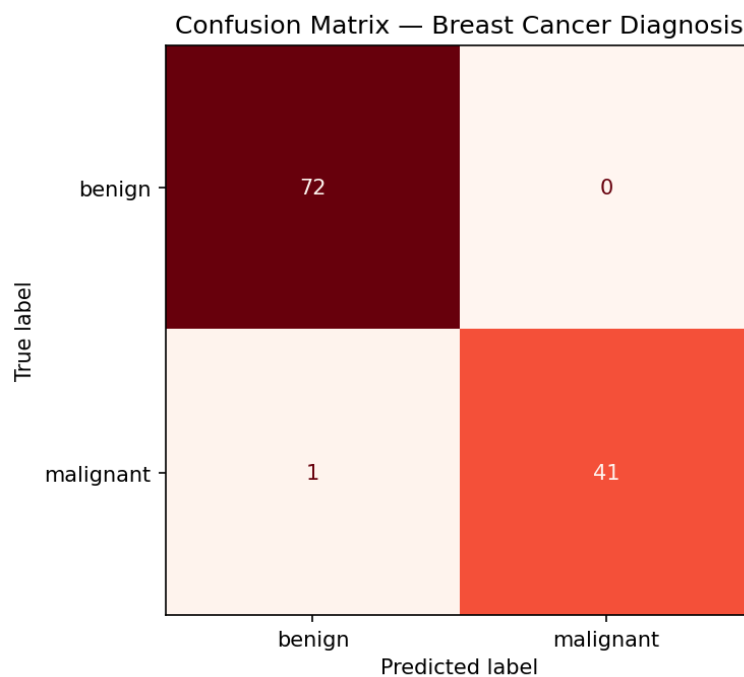
Task 5: Mini Project — Neural Network Classification

Project: Breast Cancer Classification

Accuracy & Loss



Confusion Matrix



Test Accuracy: **99.12%** | Precision (malignant): **100.00%** | Recall (malignant): **97.62%**

Project Report

Problem being solved: Classifying whether a breast tumor is malignant or benign based on 30 numeric measurements taken from digitized images of breast masses, using an Artificial Neural Network.

Dataset: The Breast Cancer Wisconsin dataset (569 patient samples, 30 features), a well-known and widely used benchmark medical dataset built into Scikit-learn.

Neural network architecture: A feedforward neural network with an input layer of 30 features, two hidden layers (32 and 16 neurons, both using ReLU activation), and a single output neuron with sigmoid activation for binary classification (malignant vs benign).

Training process: The model was trained for 60 epochs using the Adam optimizer and binary crossentropy loss, with 20% of the training data held out as a validation set to monitor performance during training.

Results: The model achieved 99.12% test accuracy, with the confusion matrix showing 72/72 benign cases and 41/42 malignant cases correctly identified (only 1 malignant case misclassified as benign). Precision for the malignant class was 100% and recall was 97.62%, meaning the model rarely raised false alarms and correctly caught nearly all actual malignant cases.

Limitations: The dataset, while a standard benchmark, is relatively small (569 samples) and was collected from a specific clinical setting, so the model's performance may not directly generalize to different patient populations or imaging equipment. The model also only uses the 30 provided numeric features and does not incorporate any imaging data directly, patient history, or genetic markers that could improve real-world diagnostic accuracy.

Future improvements: With more time, this project could explore additional architectures (e.g. more layers, dropout for regularization), use cross-validation for a more robust accuracy estimate, tune the classification threshold to better balance precision and recall for this medical use case, and validate the model on an independent, external dataset to test real-world generalization.

Report word count: approximately 280 words.

See `Mini_Project_Breast_Cancer.ipynb` for the full code, predictions, and output.